BFMR Automation

Solution Design Specification v1.6

Complete coding handoff for Google AI Studio

Compiled 10 October 2026 | Based on v1.5, the v1.6 addendum, current code, decisions and implementation evidence

This master document retains the complete v1.5 specification and v1.6 addendum, then adds a current implementation guide. Read the current-state guide first: it resolves later user changes that supersede earlier text in the preserved source documents.

Repository: /home/wardmatt/bfmr-automation/BFMR-Automation-v2

Purpose: enable another coding agent to continue implementation without relying on chat history. The source documents and working tree remain the detailed authority for code and evidence.

1. Start here: current controlling behavior

The dated amendments in this section control over conflicting requirements or historical descriptions in the v1.5 specification and v1.6 addendum. Keep every later explicit user change authoritative.

Product scope

- One qualifying single-deal BFMR email leads to a My Tracker duplicate check, a quantity-one website reservation, and a My Tracker confirmation. Successful reservations stay in place for manual release.
- Any retailer and bundles are allowed. No financial eligibility, profitability, retailer-stock, retailer-permission, purchase, cancellation, or timed release is in the current MVP 1 scope.
- Multiple distinct deals and additional-slot messages are skipped. Retailer name is not required in the email subject.
- Email trigger reservations use the deal URL in the message. A preflight duplicate, explicit full/closed/not-found state, or other qualifying skip returns to email listening.
- The email-triggered process stays open, listens serially for further qualifying messages, and reuses its headed Chromium context/page.

User-approved live-test startup reservation

- Every invocation of `npm run test-real-email` (also `test-real-email-e2e`) and `npm run test-bfmr-ipad-reservation` attempts a startup quantity-one reservation.
- Startup opens BFMR `/deals`, reads visible deal links in their existing displayed order, and does not use the In Stock filter. It checks each candidate in turn, skipping duplicates and explicitly unavailable deals. It advances after proved submission failures until My Tracker confirms a quantity-one reservation or the displayed candidates are exhausted.
- This creates real reservations. The iPad reservation test may create one startup reservation and then a second reservation for its hard-coded iPad target. The email runner reserves a startup deal and then continues listening; later email reservations are separate actions.
- Startup CAPTCHA is completed manually in Chromium. No CAPTCHA solving or bypass is used. After startup, an unexpected challenge stops the affected test and alerts rather than asking for PAGE-READY.
- No `CHECK-API` prompt remains in the live reservation/recovery scripts. The app checks My Tracker automatically after submission. Normal duplicate/unavailable email skips are not errors.

Submission safety and retries

- Create reservations only through visible BFMR website form actions. My Tracker API is read-only for duplicate checks and confirmation.
- Perform a complete My Tracker preflight before any click. A duplicate blocks an email-triggered reservation; startup proceeds to the next candidate.
- Persist startup `SUBMITTING` evidence before a possible click. On restart, reconcile an unfinished attempt through My Tracker before new startup submissions. A missing row alone does not prove that the earlier click failed.
- At most two submit ordinals per deal are permitted only after definitive proof of non-creation and a complete negative tracker read, with fresh page and duplicate checks. The current live adapter does not prove non-creation for an unqualified HTTP 500, so such outcomes remain uncertain and pause. For proved startup failures, exhaust two ordinals then advance to the next listing candidate.
- Unknown API reads, wrong quantity, identity mismatch, unexpected page, or an uncertain submission stop processing. Do not replay blindly.

- My Tracker confirmation polling for email/iPad flows is immediate, then every five seconds up to twelve additional reads while the result is `NOT_VISIBLE_YET`. Startup uses immediate plus 10-second and 20-second delayed reads, pausing for manual challenge completion when one is detected.

2. Current code, operation and handoff instructions

Runtime entrypoints

- `npm run test-real-email`: headed live runner. It captures a Gmail history baseline before startup reservation, then polls Gmail history every 15 seconds for new messages. Existing pre-baseline email is intentionally excluded by this test runner.
- `npm run test-real-email-e2e`: alias of the same script.
- `npm run test-bfmr-ipad-reservation`: startup reservation followed by the existing hard-coded iPad Air live test.
- `npm run resume-watch-series-reservation`: legacy/manual recovery test; now has automatic API confirmation and still requires explicit pre-action PAGE-READY identity review.
- `npm run auth-bfmr`, `npm run auth-gmail`: interactive account/session setup helpers. They are not startup-reservation flows.
- `npm test`: offline automated test suite; fakes and synthetic browser fixtures only, no real inventory mutations.
- `npm run typecheck`, `npm run build`: TypeScript check and app build.

Configuration and sensitive data

Live test runners load local environment configuration through `scripts/loadEnv.ts`. Required setting names include:

- Gmail: `GMAIL_CLIENT_ID`, `GMAIL_CLIENT_SECRET`, `GMAIL_REFRESH_TOKEN`; optional `GMAIL_ACCOUNT_EMAIL` for readable intake logs.
- BFMR: `BFMR_EXPECTED_ACCOUNT`, `BFMR_API_KEY`, `BFMR_API_SECRET`.
- Telegram: `TELEGRAM_BOT_TOKEN`, `TELEGRAM_CHAT_ID`.
- Optional startup journal path: `BFMR_STARTUP_STATE_PATH`; default is `~/.local/state/bfmr/startup-reservation.json`, with transition audit at the adjacent `.events.jsonl` path.

The environment loader checks a configured `BFMR_ENV_FILE`, repo `.env`, `~/.env.bfmr_automation`, then `~/.env`, with existing shell values taking precedence. Google credentials may be read from `~/.bfmr/gcp-credentials.json`. Never put credentials, tokens, raw email, account data, cookies, browser profiles or tracker rows into this PDF, logs, prompts or Git. No secret values are included here.

Source map for continuing implementation

- `src/adapters/playwrightBrowser.ts`: headed persistent Chromium session, trusted navigation, visible deal listing reader, form submission, challenge detection, title/unavailable-page adapter.
- `src/services/startupReservation.ts` and `src/services/startupJournal.ts`: startup candidate policy, journaled retries and restart reconciliation.
- `src/services/bfmrTrackerApi.ts`: read-only complete/paginated My Tracker checks. Never convert incomplete/unknown results to absence.
- `src/services/reservationAlerts.ts` and `src/services/automaticConfirmation.ts`: Telegram failure reporting and prompt-free confirmation polling.
- `src/services/bfmrEmailLinks.ts`, `emailParser.ts`, `bfmrUrl.ts`: local decoding of BFMR tracked email links, single-deal qualification, trusted destinations.

- `src/services/dealAvailability.ts`, `visibleDealTitle.ts`: explicit unavailable-state and rendered catalog heading recognition.
- `src/services/browserSession.ts`: browser lifetime/startup hook and email processing loop.
- `scripts/test_real_email_e2e.ts`, `scripts/test_bfmr_ipad_reservation.ts`, `scripts/live_test_startup.ts`, `scripts/resume_watch_series_reservation.ts`: live script wiring.
- `src/orchestrator/engine.ts`, `src/db/schema.ts`, `src/db/migrations/001_initial_schema.sql`: separate design/orchestration scaffolding. The user-facing live scripts are not yet a completed production durable service.

3. Verified state and work remaining

Evidence recorded as of 10 October 2026

- The offline suite passed 59/59 tests. Project TypeScript checks, explicit TypeScript checks for live entrypoint scripts, and `git diff --check` passed.
- Synthetic Chromium tests cover visible title reading and ambiguity, full/not-found messages, listing-route distinctions, displayed deal ordering, and form/browser state behavior.
- Synthetic email fixtures cover forwarded NEW DEAL / FULL RETAIL subjects, tracked redirect decoding, multiple deals, malformed payloads and unsafe destinations.
- Telegram transport tests use an injected fake to test request construction, configuration, response validation and redaction. After those checks, the real configured bot also sent one clearly marked test alert using a simulated reservation failure; the original simulated exception remained intact. No BFMR action occurred during that notification test.
- No live BFMR startup reservation or full live end-to-end run was performed as part of document preparation. User-run tests have exposed actual filter-locator and multiple-heading title issues; code was adjusted, but exact current BFMR DOM behavior still needs live confirmation.

Known gaps before claiming production readiness

- The real BFMR In Stock locator was removed per user direction. The unfiltered visible link extraction and current first-deal order must be verified against the live listing.
- Live CAPTCHA timing, listing order, deal title/price markup, quantity form identity, exact selected variant, and My Tracker immediate visibility remain integration evidence tasks. The automation stops on unresolved identity.
- `test-real-email` polls Gmail every 15 seconds and takes a fresh history baseline at each process start; messages before that baseline are excluded. It is not durable catch-up across restarts and does not use the production Pub/Sub pipeline.
- Startup recovery uses an owner-only local JSON snapshot and JSONL transition audit, not the PostgreSQL attempt journal/outbox in v1.5.
- Telegram delivery was tested with one real test-only message. The live-script sender is immediate and does not durably queue/replay a failed Telegram delivery. The production notification outbox and blocked gate remain unfinished.
- The repository includes in-memory/UI orchestration scaffolding; do not present it as a deployed or validated PostgreSQL production service.
- MVP 2 is only a historical roadmap label for post-reservation evaluation. Its criteria and cancellation behavior are deferred and unapproved; do not implement them without new user decisions. The source spec labels MVP 3 purchasing/order update, MVP 4 shipment tracking, MVP 5 additional slots, MVP 6 multiple-deal mail; reconcile before use.

Instructions to the next coding agent

Read this current-state section and then the entire preserved v1.5 specification and v1.6 addendum. Inspect the live working tree before editing; preserve user changes. Treat later explicit user instructions as controlling. Keep UI/API boundaries, durable evidence, retries and privacy safeguards intact. Do not use real BFMR inventory or send real Telegram messages during offline validation. Never claim live validation from synthetic tests. Implement the production durability and integration gaps only as required by the user, updating design notes and evidence as each item changes. The package runs in the provided local checkout; maintain the source files as well as this PDF when updating requirements.

Document contents and precedence

1. This current handoff and amendment summary (controlling for the later user changes stated here).
2. Complete v1.5 PDF specification, preserved byte-for-byte as source pages.
3. Complete v1.6 Markdown addendum and dated amendments (retained as history; current amendments above supersede earlier addendum text where they conflict).
4. Decision register, implementation plan and open issues as supplementary project evidence. These are informative and must be read with the precedence rules above.

Deferred roadmap labels from the v1.5 source

Milestone label	Historical roadmap description	Current status
MVP 2	Post-reservation evaluation	Deferred; not approved or fully specified.
MVP 3	Purchasing and order update	Deferred; not approved.
MVP 4	Shipment tracking	Deferred; not approved.
MVP 5	Additional-slot emails	Deferred; not approved.
MVP 6	Multiple-deal emails	Deferred; not approved.

Do not infer activation from the roadmap names. Reconcile requirements with Matt before implementing any later milestone.

Paste-ready continuation prompt

You are continuing the existing BFMR Automation v2 repository at `/home/wardmatt/bfmr-automation/BFMR-Automation-v2`, not starting a new project. Read this complete v1.6 handoff and inspect the repository and working tree before editing. Follow the current-state section first when older source text conflicts; preserve all existing user changes. Implement only the current MVP 1 requirements or new explicit user requests. Do not treat deferred MVP 2-6 roadmap labels as approval. Keep reservations on the visible BFMR website flow, use My Tracker read-only for duplicate/confirmation checks, and preserve durable evidence and conservative retry behavior. Do not run live BFMR reservation commands or send Telegram messages unless Matt explicitly requests it. Keep secrets and private account/email data out of prompts, logs, fixtures and Git. Report what changed, what was tested and what still needs live validation.

Purpose and authority

BFMR MVP 1 Solution Design Specification

Version 1.5 | 9 October 2026 | Prepared for Matt Ward

Receive a single new-deal email, identify the BFMR deal in an authenticated browser, check My Tracker for an existing active reservation, and reserve quantity 1 through the BFMR website. Confirm the result through My Tracker. Matt releases reservations manually.

Current scope

All retailers and bundle deals are permitted. MVP 1 performs no financial, profitability, retailer-stock or retailer-eligibility evaluation and makes no purchases. Multiple-deal emails and additional-slot announcements are ignored. Testing and production both make real reservations.

Authority and status

This specification replaces v1.4 for MVP 1 and incorporates the final decisions in the accompanying inconsistency-review conversation. Later explicit user answers override earlier answers, including intermediate choices withdrawn during that conversation. Earlier source documents are historical references where they conflict with this version.

Confirmed requirements are identified as such. Retained engineering decisions and new engineering consequences are distinguished from user requirements. This is an implementation design, not evidence that the system has been implemented or validated live.

Reading guide

Pages	Contents
2-4	Change register and requirements
5-10	Architecture, intake, browser/API contracts, state and recovery
11-15	Notifications, persistence, concurrency, configuration and operations
16-18	Acceptance tests and implementation gates
19-21	Deferred decisions, source defects and traceability

Resolved decisions and precedence

Topic	Authoritative v1.5 disposition
Reservation flow	Reserve quantity 1 after trigger/identity, duplicate and operational safety checks. No business-eligibility evaluation before or after reservation.
Retailers and bundles	Any retailer; bundles included. Amazon-only restrictions and bundle rejection are superseded.
Release and purchase	All releases are manual. No one-minute release, failed-eligibility release, cancellation API calls or retailer purchase in MVP 1.
Duplicate safeguard	Any existing active reservation for the same deal blocks a new reservation, including a manual reservation and any color.
Email scope	Single new-deal emails only. Ignore multiple-deal emails and additional-slot notifications. A bundle offer is one deal, even if it contains several products.
Subject syntax	Accept both historical NEW DEAL | retailer subjects and variants such as DEAL DAYS - NEW DEAL. Retailer channel is not required.
Downtime	Catch up and process qualifying emails received while stopped. Preserve the durable Gmail cursor; remove restart receipt-time exclusions.
Platform	Single Ubuntu host with Playwright bundled Chromium in headed mode; secured visible session for manual login/MFA.
Deal navigation	Open the validated BFMR email link directly in the dedicated authenticated browser. No mandatory Active Deals lookup first.
External operations	Playwright website form is the only reservation-creation route. My Tracker API performs duplicate and confirmation reads.
Testing	Both testing and production create real reservations. No default dry-run requirement; offline tests still use fake adapters.
Financial discussion	All financial decisions are excluded from MVP 1. Page 19 preserves them as deferred discussion, not instructions to implement.

Safety checks are not business eligibility

Email provenance, correct deal/account identity, duplicate prevention, quantity selection, reliable storage and uncertain-result reconciliation remain necessary to perform the authorized action correctly. They do not filter on retailer, price, payout, stock or bundle status.

Functional requirements

Stable FR IDs from v1.4 are retained with revised meanings where the user changed scope. FR-012 explicitly records manual release and the absence of purchasing.

ID	Confirmed required behavior
FR-001	Monitor one Gmail Inbox. Direct sender: deals@buyformeretail.com. Testing additionally permits forwards from matt.ward@live.com. Both modes create real reservations.
FR-002	Process single new-deal emails. Ignore multiple distinct deals, additional-slot announcements and unrelated mail. Do not require a retailer channel in the subject.
FR-003	Reserve regardless of retailer and include bundles. Do not evaluate financial criteria, Amazon stock, retailer permission or purchase eligibility. No maximum response-time guarantee.
FR-004	Read My Tracker for the configured BFMR account. Any matching deal_id in reserved status blocks creation, including manual reservations and all variants. A later different email can qualify after release.
FR-005	Submit quantity 1 total for one deal offer, not one unit per color. Any available color is acceptable. Preserve selected variant/color or bundle-offer identity for reconciliation.
FR-006	One active deal worker. Process oldest known qualifying email first. Failed or incomplete duplicate-status reads hold the queue until a complete read succeeds.
FR-007	Catch up after downtime and retain admitted unfinished work across crashes. Deduplicate delivery and processing; do not exclude mail merely because receipt preceded the current process start.
FR-008	Reconcile uncertain submissions, including after restart. Unresolved outcomes create review and Telegram alert; only the affected deal is blocked once uncertainty is classified.
FR-009	Confirmed non-creation permits one automatic retry. Second failure stops that deal and alerts. Closed/no-slot results are skipped without retry or alert.
FR-010	Local terminal commands resolve reviews and explicitly request reprocessing. Resolution does not submit. Reprocessing grants one initial attempt and one retry after safety checks.
FR-011	Duplicate delivery never automatically replays a completed email. New mail cannot bypass current-run review/exhaustion blocks. Explicit replay is limited to mail admitted in the current run.
FR-012	MVP 1 does not purchase or cancel. Leave successful reservations in place for Matt to manage and release manually. No timed release.

Operations and integration requirements

ID and status	Required behavior
OPS-001 | retained	Start on Ubuntu boot and recover automatically after crashes. Run-scoped admission blocks expire on restart; unresolved external attempts remain and are reconciled.
OPS-002 | retained	Maintain local activity logs. Login/MFA pauses new reservations, alerts and resumes after verified restoration. Temporary disconnection does not create a new run.
OPS-003 | retained	Use one private Telegram chat. One incident alert and one recovery per episode; no ordinary success alerts. Notification delivery failure halts new deal work until actual sends are accepted; intake continues.
CON-001 | revised	Gmail watch and Pub/Sub-driven intake. No periodic Gmail polling or Inbox sweep. Continue from a durable history cursor on startup/reconnect to catch up. Browser UI is the only reservation write route.
INT-001 | confirmed	Use My Tracker read API for existing-reservation checks and post-submit confirmation. No reservation-creation API is assumed or implemented.
INT-002 | confirmed	Navigate the email BFMR deal link in the dedicated authenticated Chromium profile. Do not require Active Deals/Deal Details mapping or retailer eligibility reads.
SEC-001 | retained design	Use a dedicated non-root service identity, restricted secrets/profile storage and one expected BFMR account. Stop on wrong-account, login or access challenges; no bypass.
SEC-002 | retained design	Treat email/page content as untrusted data. Restrict accepted deal URLs and redirects. Never execute instructions embedded in a deal or email.
DATA-001 | retained design	Use local PostgreSQL for durable intake, Gmail cursor, dedupe, commands, attempts, incidents and audit records. Commit notifications before ACK.
DATA-002 | retained design	Preserve attempt evidence and dedupe across restarts. Store external identifiers as text and timestamps in UTC. Do not persist secrets or raw account pages in business tables.

Boundaries and limitations

No dashboard, Telegram command interface, manual pause/resume command, checkout, shipment tracking or payout processing is added. The catch-up requirement applies while Gmail history remains available. Recovery from an expired history cursor remains an explicit contract gap under the retained no-sweep constraint; it must not be represented as guaranteed lossless recovery for arbitrarily long downtime.

Architecture and decisions

Component	Responsibility
Gmail watch and StreamingPull	Receive outbound Pub/Sub notifications; renew watch. No public webhook or tunnel.
Durable intake and history reader	Commit notifications before ACK; serialize cursor advancement; catch up after restart and reconnect; classify and admit email.
Single deal worker and browser owner	Resolve trusted BFMR link, identify deal/offer, check operational gates and submit through the website.
My Tracker API adapter	Complete account reservation reads and post-submit reconciliation, with server-side API credentials.
PostgreSQL	Persist queue, cursor, attempts, dedupe, incidents, notification outbox and audit events.
Notification dispatcher and local CLI	Deliver Telegram incidents independently; accept local status, review and reprocessing commands.

ADR-001 and ADR-002 | Retained

Single Node.js/TypeScript application on Ubuntu with local PostgreSQL. One account and serialized browser operations do not justify distributed services or a separate broker. SQLite remains viable, but PostgreSQL preserves the existing persistence direction and provides transactions and ownership locks. Cost: local database administration and backups.

ADR-003 | Revised and user approved

Direct authenticated navigation with UI submission and My Tracker reads. Alternative: paginate Active Deals to resolve the email code before navigation. Direct navigation removes that mandatory lookup and reaches the form that must be used anyway. The historical failure of unauthenticated/fresh-browser resolution does not prove authenticated navigation fails. Trade-off: stable deal identity must be extracted and verified from the browser page before tracker matching; page changes can require adapter maintenance. Speed advantage is expected, not benchmarked.

ADR-004 and ADR-005 | Persistence and catch-up

Durability does not authorize replay of completed work. Retain event evidence, per-command attempt budgets and run-scoped blocks separately. Revised design: persist the Gmail cursor across starts and perform one startup continuation after activating the listener. First-ever initialization and expired-cursor handling are explicit engineering boundaries on page 6, not silent restart cutoffs.

Gmail intake and downtime catch-up

Startup sequence

1. Validate configuration and Gmail identity; open PostgreSQL and obtain singleton ownership. Create a STARTING run. Preserve the existing consumed history cursor. Keep submissions behind the recovery barrier while starting independent intake.

2. Establish or renew the INBOX watch; record watch expiration and watch baseline separately from the consumption cursor. Start StreamingPull and atomically activate the run. Buffer callbacks until activation is committed. listener_started_at is an audit timestamp, not an email eligibility cutoff.

3. When a saved cursor exists, perform one startup history continuation from that cursor without waiting for a new notification. Process changes from downtime, then continue event-driven reads. Renewing the watch must never overwrite the saved consumption cursor.

First-ever initialization | Engineering decision

With no prior cursor and no restored history state, save the initial watch baseline as the initial consumption cursor, then continue from it. This begins monitoring from first setup; it does not scan historical Inbox contents. An existing installation missing its cursor is a recovery fault, not permission to silently initialize afresh. Historic mail before first setup is not authorized by the downtime catch-up decision.

Durable processing

Validate notification mailbox/shape. Insert under unique subscription/message ID, then ACK after commit. Verify existing duplicate records before ACK. Persist sanitized rejected-event evidence before acknowledging poison input. Do not ACK while persistence is unavailable.

Read Gmail history serially and consume all pages. Follow message-added and relevant Inbox-label-added events; fetch each distinct message and require INBOX in its fetched labels. Retain the prior v1.4 behavior for Inbox membership. Sorting is internalDate, then Gmail ID. Atomically persist classifications/admissions and advance the fully consumed checkpoint. Any required fetch/page failure leaves the cursor unchanged; database dedupe makes repetition harmless.

Reconnect, watch maintenance and history gaps

Temporary interruptions retain the run and cursor. Perform a one-time reconnect continuation, then resume event-driven reads; no quiet-mailbox polling. Renew watch daily and retain expiration separately. Delayed notifications may reveal earlier missed changes; no-poll operation cannot promise prompt discovery.

On an expired/rejected history cursor, raise GMAIL_HISTORY_GAP, retain the cursor and stop new intake. Restart must not erase the gap or silently reset the baseline. Already-admitted work may proceed if other gates permit. A full Inbox recovery sweep remains outside this design; guaranteed recovery beyond history retention requires a further explicit decision. Validate actual Gmail retention behavior before release.

Classification and deal identity

Sender and message classification

Production accepts direct mail from deals@buyformeretail.com. Testing also accepts forwards whose outer From mailbox is matt.ward@live.com. Normalize mailbox casing, not display names. Embedded forwarded headers are descriptive content, not proof of sender identity. Use trusted Gmail authentication evidence with fixture-backed rules; unknown authentication for a recognized candidate produces no reservation and a diagnostic alert.

Handle repeated Re/Fw/Fwd prefixes. Accept historical NEW DEAL | retailer and DEAL DAYS - NEW DEAL layouts without a required retailer channel. Preserve HTML hyperlink targets when plain text omits them. Count distinct deal identities, collapsing repeated links to the same deal; navigation/banner links do not count. Unambiguous multi-deal mail and additional-slot announcements are silent logged skips. Ambiguous single-deal identity creates review.

Direct BFMR navigation

Accept only HTTPS deal links on an exact approved BFMR hostname/path (initial known hosts: bfmr.com and www.bfmr.com; /deals/ path). Parse URLs, never use substring host matching. Reject embedded user credentials, unexpected ports and arbitrary destinations. Restrict top-level redirects to validated BFMR destinations; any necessary external login destination requires a separately verified authentication contract before use.

Navigate the email link in the dedicated authenticated browser. Verify expected account and establish stable BFMR deal_id, canonical page URL and the correct reservation form. Keep email deal code, canonical slug and deal_id distinct; never substitute the short URL token for an API slug or tracker ID. If the mapping cannot be established reliably, stop that deal for review. Active Deals API mapping is not a required step or automatic fallback.

Offer and quantity selection

Retailer, retail stock, price and payout do not gate MVP 1. Bundles are allowed. Interpret quantity 1 as one BFMR offer/reservation unit, which may contain multiple bundle components; do not reserve each component separately. This is an engineering interpretation of the confirmed bundle inclusion and quantity rule and must match the real form contract.

For alternative colors/variants choose one available reservable offer using stable variant ID order, or normalized label and stable form position if IDs are absent. Persist selected offer identity, color/description where applicable, and bundle-component identity when necessary to distinguish offers. Lack of retailer stock is not a reason to reject a deal. BFMR closed/no-slot availability is an execution constraint and produces a silent skip.

Browser and My Tracker contracts

Operation	Required result and behavior
verifySession(account)	AUTHENTICATED with expected identity; AUTH_REQUIRED, CHALLENGE or UNKNOWN otherwise. Wrong account blocks all submissions.
readDeal(candidate)	IDENTIFIED: stable deal_id, canonical page, reservation form and offered variants/bundle identity. No financial/retailer/stock eligibility result. Alternatives: UNAVAILABLE, AUTH_REQUIRED or REVIEW.
inspectReservations(account, deal)	GET My Tracker with API-KEY and API-SECRET. Match deal_id across all variants. Any reserved row means RESERVED. Return CONFIRMED_NONE only after complete, fresh, validated pagination; errors/ambiguity mean UNKNOWN.
submitOne(attempt, offer)	Normal website-form action, quantity 1 for exactly one offer. Return acknowledgement evidence or UNCERTAIN. Never internally repeat the click or request after submission may have begun.
reconcile(attempt)	CONFIRMED_ONE, CONFIRMED_NOT_CREATED, CONFLICT or UNKNOWN. Verify deal, quantity, offer/variant and RID where present. AUTH_REQUIRED causes global pause.

Tracker read contract and evidence

Use the supplied GET /api/v2/my-tracker endpoint with explicit date bounds covering at least the preceding 24 hours plus a configurable boundary buffer; consume every page. The 24-hour minimum derives from reported purge behavior, not the published response schema. Verify timezone, date-field meaning, pagination, status and RID semantics before live use. If older active reservations can exist, widen/remove the date filter so absence is never inferred from an incomplete window.

The published example contains empty objects and does not establish a complete response schema. Preserve raw field-name mappings only in sanitized fixtures. Check that API credentials and browser session refer to the same BFMR account. Separate transport, authentication, schema and incomplete-page failures from empty results.

Proof of creation and non-creation

A success toast is acknowledgement only. Final confirmation requires My Tracker to show the expected deal, quantity 1 and matching offer identity. Wrong quantity, mismatched offer or possible simultaneous manual creation is CONFLICT; do not cancel or modify it.

CONFIRMED_NOT_CREATED requires a definitive rejection or a complete, fresh tracker read under a validated immediate-visibility contract. The reported immediate appearance and manual-release removal require live evidence. If those guarantees cannot be established, absence after an uncertain click remains UNKNOWN and cannot authorize a retry. Read-only API keys stay outside the browser.

Command states and retry limits

State	Transition and effect
QUEUED	Claim oldest runnable command, check run/deal blocks, then CHECKING. One account worker.
CHECKING	Verify session and deal/offer identity, then complete duplicate check. Qualifying action -> READY; existing reservation/closed/no-slot -> SKIPPED; ambiguous identity -> REVIEW.
WAIT_STATUS	Failed or incomplete reservation-status read holds the queue. Retry reads, alert once and recover once; no submit attempt consumed.
READY	Recheck global gates and fresh absence. Commit next attempt as SUBMITTING before invoking submitOne.
SUBMITTING	One external invocation for this ordinal. Record available outcome, then RECONCILING even after apparent success.
RECONCILING	Target reads near 0, 10 and 30 seconds. Confirmed one -> RESERVED; proved non-creation -> RETRY_PENDING or EXHAUSTED; unresolved/conflict -> REVIEW.
RETRY_PENDING	After confirmed first-attempt non-creation only. Refresh identity, availability, gates and duplicate status before ordinal 2.
RESERVED / SKIPPED	Terminal. No automatic replay on duplicate delivery; no reservation-success notification or automatic release.
REVIEW / EXHAUSTED	Terminal automatic outcome. Current-run deal block and incident. Unrelated deals proceed once global blockers are clear.

Attempt budget

Maximum two submission ordinals per command: initial plus one retry. Read retries do not consume this budget. A possibly issued action does. A crash between committing SUBMITTING and the click is still uncertain and requires reconciliation. Restart never resets an unfinished command budget. No hidden submit retry inside generic browser or network helpers.

Closed/no-slots proved before or by a definitive submit response becomes SKIPPED_UNAVAILABLE with no retry/alert. A timer never turns an unknown outcome into non-creation. An explicit eligible CLI replay creates a new command and budget only after review resolution and live checks.

Scheduling

Order by original Gmail internalDate, Gmail ID and monotonic command sequence. Late-discovered downtime mail cannot undo a completed reservation. Finish or classify the active command before moving on. Manual replay retains its email priority without interrupting work. Application serialization cannot prevent a manual BFMR action between check and click; classify observed conflicts conservatively.

Restart recovery and operator controls

Persisted situation	Restart behavior
QUEUED / CHECKING / READY / WAIT_STATUS	Keep admitted work; restart checks using live facts. Preserve original admission and email receipt time; execute in the new run.
SUBMITTING / RECONCILING	Reconcile before any new submission. Confirmed matching creation -> RESERVED; proved non-creation -> remaining retry only; uncertainty/conflict -> current-run review block.
RETRY_PENDING	Retain attempt count and perform ordinal 2 only after fresh checks.
RESERVED / SKIPPED / REVIEW / EXHAUSTED	Do not automatically re-enqueue terminal commands. Expire old run-scoped admission blocks, but recreate blocks for unresolved external effects.
Email received during downtime	Continue the saved Gmail history cursor and admit qualifying unprocessed mail, regardless of listener start time. Apply ordinary dedupe and deal blocks.
Temporary outage / mode change	Outage retains run. Real restart may change mode; re-evaluate sender permission on unfinished backlog. Testing forwards become SKIPPED_MODE in production.

Recovery barrier

No new submission until every potentially issued unresolved attempt is safely resolved or isolated under a current-run review block. If account-status inspection itself fails, hold the queue. Intake and essential recording may continue. A restart is never a substitute for incident resolution. Telegram failure and Gmail history-gap state persist independently of run-scoped deal blocks.

Local CLI contract | Retained design

Command	Semantics
bfmr status / bfmr activity	Read mode, cursor/catch-up state, queued/carried work, active command, gates and sanitized history.
bfmr reviews list	List current-run review/exhaustion incidents; historical records remain read-only.
bfmr review resolve <id>	Require outcome, evidence/note and expected version. Outcomes: confirmed-created, confirmed-not-created, acknowledged-failure for known non-creation. Enqueue nothing.
bfmr emails list	Show admitted mail, retained backlog, terminal state and replay eligibility.
bfmr email reprocess <gmail-id>	Require current-run admission, no unresolved review and no active command. Create one idempotent CLI command with fresh two-attempt budget.

Downtime mail may enter the current admission run. Completed prior-run mail cannot be replayed by CLI. No cancellation or pause/resume command.

Telegram incidents and gates

Incident policy

Send one alert when a new incident opens and one recovery when the condition is proved resolved. Incidents include Gmail authorization/watch/history gaps, BFMR login or wrong account, failed reservation-status read, unidentifiable single deal, uncertain/conflicting submission and exhausted retry. Ordinary skips and successful reservations do not send alerts. Known multi-deal, extra-slot, existing-reservation and closed/no-slot outcomes are logged silently.

Acceptance controls recovery

Use sendMessage to the fixed private chat. Delivery acceptance requires a valid response with ok=true and a returned Message. HTTP reachability, DNS, getMe or a bot identity probe is not enough. Do not send a proactive startup test. Persist a known delivery failure before continuing deal work; retry the failed outbox item with backoff and Retry-After.

After accepted delivery, send the Telegram recovery message and drain required pending operational messages. Clear TELEGRAM_BLOCKED only after those sends are accepted. Intake, durable recording and essential reconciliation remain active while new identity checks/submissions are halted. If a click already began, finish recording/reconciling it; never interpret the new gate as proof nothing happened.

Gate	Clears when
PERSISTENCE_UNHEALTHY / OWNERSHIP_LOST	Storage and exclusive ownership are healthy and all possible in-flight effects reconciled.
STARTUP_RECONCILIATION	Every possible prior submission is resolved or isolated as review.
BFMR_AUTH_REQUIRED	Expected BFMR account identity/session verified after manual login/MFA.
RESERVATION_STATUS_UNAVAILABLE	Complete fresh status check succeeds for the blocked command.
TELEGRAM_BLOCKED	Pending failed notification, required recovery and pending operational messages are accepted.

Failure semantics

All active submission gates must clear before new work proceeds. Gmail intake loss alone does not invalidate already-durable work; failure to deliver its alert may independently halt it through TELEGRAM_BLOCKED. This retains the v1.4 operational design.

Accepted sends with lost responses may be retried and duplicated. Include a stable incident ID; never claim exactly-once Telegram delivery. Alert/recovery phases have unique outbox keys. Review/exhaustion incidents are resolved by a supported operator disposition, not timeout. Notification acceptance is not proof that Matt read the message.

Data model and invariants

Internal UUIDs; external IDs as text; UTC timestamps; JSONB only for bounded sanitized evidence. All commands retain email and account references. Browser/API secrets are never stored in business records.

Table	Core fields and invariants
runs	id, mode, created_at, listener_started_at, ended_at, config_version, status. Mode immutable while active. Start time is audit only, not admission cutoff.
inbound_events	id, subscription, provider_message_id, mailbox, history_id, run_id, state. UNIQUE(subscription, provider_message_id).
gmail_state	mailbox PK, consumed_history_id, watch_baseline_history_id, watch_expiration, last_success_at, intake_state, initialization marker. Cursor persists across runs; one writer.
emails	id, mailbox, gmail_id, internal_date, outer_sender, auth_result, inbox_at_fetch, admission_run_id, classification, candidate_refs, content_hash. UNIQUE(mailbox, gmail_id).
deal_observations	id, account_id, deal_id, email reference/code, canonical URL/slug where known, offer/variant identity, color, optional bundle components, observed_at. No eligibility evaluation fields required.
commands	id, email_id, admission_run_id, execution_run_id, AUTO/CLI kind, request_id UNIQUE, state, attempt_count 0..2, queue_time, monotonic sequence, deal_id.
attempts	id, command_id, ordinal 1..2, account/deal, selected offer identity, quantity CHECK=1, prepared_at, invoked_at nullable, result, reservation_id, evidence. UNIQUE(command_id, ordinal).
deal_blocks	id, run_id, account_id, deal_id, REVIEW/EXHAUSTED type, reason, resolved_at. Unique active block per run/account/deal/type.
incidents	id, scope, type, episode, ACTIVE/RESOLVED state, opened_at, resolved_at, disposition. One active incident per scope/type.
notification_outbox	id, incident_id, ALERT/RECOVERY phase, state, due_at, tries, telegram_message_id. UNIQUE(incident_id, phase).
runtime_gates / workflow_events	Gate name, reason, since; append-only events with actor, correlation/causation IDs, run/command/attempt IDs and sanitized detail.

Indexes and boundaries

One AUTO command per email; one active command per email across AUTO/CLI. Index queue ordering, unresolved attempts, active deal blocks and due notifications. Do not enforce permanent one-reservation-per-deal: manual release followed by a different email is valid. Review evidence survives expiry of run-scoped blocks. Financial, order, shipment and payout tables are outside MVP 1.

Transactions and ownership

Boundary	Atomic commit and crash rule
T1 Receipt	Persist validated/rejected notification, then ACK. Redelivery finds the unique key.
T2 History batch	Persist classifications, email dedupe and command admissions with the fully consumed cursor. Failed fetch/page leaves checkpoint unchanged.
T3 Attempt preparation	Lock command; check budget/gates; allocate ordinal; commit SUBMITTING with chosen offer and pre-submit evidence. No external click inside the transaction.
T4 Outcome	Record evidence, command/attempt result, deal block, incident, notification and audit event atomically.
T5 Operator action	Validate request ID and expected entity version; record resolution or enqueue one CLI command with audit.
T6 Notification	Record accepted Telegram ID/phase and clear gate only after required accepted sends. Lost response leaves uncertain delivery.

Exclusive browser ownership

Use a process singleton lock plus a dedicated PostgreSQL advisory lock for the BFMR account. A second instance cannot start a listener run or submit. One browser owner serializes all page operations; health/auth probes and the CLI cannot race the worker. The CLI submits requests to the worker and never drives the browser or mutates tables directly.

systemd must terminate the old process group, including Chromium, before replacement. Reacquiring a database lock alone is not proof an old browser stopped. On connection/lock-health loss, cease initiating browser actions immediately. The supervisor terminates an unresponsive owner; recovery reconciles possible side effects. Multiple hosts sharing the same BFMR account are unsupported.

External uncertainty

A local commit and BFMR form submission are not atomic. A crash after T3 may occur before or after the click. Reconcile both cases. The design gives one application submit invocation per recorded attempt while ownership is healthy, not a mathematical exactly-once guarantee from BFMR. Three reads are a bounded investigation, not permission to retry after 30 seconds.

Backpressure

Bound Pub/Sub in-flight delivery and Gmail fetch concurrency. ACK only durable notifications. Storage pressure delays ACK safely and may cause duplicate delivery. Raise an incident and stop new effects before storage becomes unusable. Keep the authoritative queue in PostgreSQL, never solely in memory. Audit failures must not cause an unrecorded blind retry.

Configuration and local interfaces

Configuration	Contract
Mode and identities	MODE=testing|production is mandatory and immutable until restart. GMAIL_ACCOUNT_EMAIL and BFMR_EXPECTED_ACCOUNT are required. Validate Gmail OAuth and browser/API BFMR account alignment.
Gmail and Pub/Sub	Configured project/topic/subscription and credential references. Gmail refresh credentials, watch expiration and persistent cursor are distinct concerns.
BFMR API	BFMR_API_KEY and BFMR_API_SECRET references are required for My Tracker reads. Keep server-side. Reservation UI does not eliminate tracker authentication requirements.
Browser	Pinned Playwright version and matching bundled Chromium; dedicated persistent profile path; headed mode and secured desktop session. No Edge dependency.
Database and logs	Local database connection secret reference, restricted log directory, backup destination and retention. Configuration/schema version recorded with runs.
Telegram	One fixed private chat ID and secret bot-token reference. No incoming Telegram command endpoint.
Read controls	Tracker date-window/buffer settings, validated pagination contract, finite API/browser timeouts and retry policy. No business eligibility thresholds used in MVP 1.

Defaults | Engineering decisions

Renew Gmail watch daily. Read/notification retries begin at 5 seconds and back off exponentially to 60 seconds with jitter; honor Retry-After. Reconciliation targets 0/10/30-second reads but must respect validated provider consistency. Unknown submit outcomes never flow through generic retries. Keep ordinary logs/intake evidence for 180 days; preserve unresolved effects and minimal dedupe longer.

CLI transport

Use a permission-restricted Unix socket under /run/bfmr. Each request carries schema version and request_id; repeating the same request ID returns the same result. Reject unknown fields. Resolve/reprocess operations require expected version checks. Stable errors: NOT_CURRENT_RUN, REVIEW_UNRESOLVED, ALREADY_ACTIVE, UNKNOWN_ID, NOT_AUTHORIZED and SERVICE_UNAVAILABLE.

Removed or deferred settings

No one-minute cancellation timer, financial limits, Amazon stock gate, retailer allowlist or dry-run default is part of the running MVP 1 configuration. The trusted BFMR hostname list is a security control, not a retailer allowlist. Test fakes are dependency-injected in automated tests; the testing deployment mode still reserves real inventory.

Security and Ubuntu operations

Boot and desktop session

Run under a dedicated non-root account using systemd with automatic boot start, restart-on-failure and backoff. Depend on local PostgreSQL/network readiness; expected outages should retry within the live process. Use a dedicated persistent Chromium profile restricted to the service owner. Pin Playwright and install its matching browser from the actual application repository.

Headed Chromium requires a display session, not a physical monitor. Provide a secured desktop/remote desktop so Matt can complete login/MFA. A virtual display alone is not an operator interface. Missing session access means a pause, not invisible unattended login. Do not silently enable insecure autologin or expose debugging ports. Validate startup after reboot and session recovery on the Ubuntu host.

Credentials and external content

Scope Gmail to required read/watch access and Pub/Sub to the configured subscription. Keep OAuth, BFMR keys, Telegram token, database credentials and browser sessions outside Git, artifacts, logs and AI inputs. Restrict local secret files. Do not send BFMR API headers into browser automation. Configure API base URLs explicitly; never derive them from an email.

Verify trusted senders and permitted navigation destinations. Do not log token-bearing URLs, authorization headers, cookies, full email bodies, raw account pages or reusable credentials. Stop for access challenges; no stealth, evasion or MFA bypass. Do not expose public webhook, database or remote-desktop ports.

Retention, backups and restore

Retain ordinary logs and intake evidence for 180 days. Keep unresolved attempt/review evidence until disposed and minimal email/command dedupe tombstones beyond payload expiry. Keep selected offer/reservation identity for audit and future purchase matching; define longer archival policy before expansion.

Take encrypted daily PostgreSQL backups off-host and test restoration. Keep secret recovery material separately. Record snapshot/config/schema versions. A stale backup may omit actual submissions: reconcile the account before scheduling and hold affected work when recent actions cannot be reconstructed. A lost Gmail cursor is an incident, not permission to reset and discard missed mail.

Operational failures

Database failure blocks submission and ACK of unpersisted messages. Attempt sanitized fallback logs and alerts where possible. Preserve known Telegram failure and history-gap incidents across restart. Configure power recovery and monitor disk space/watch renewal. Development remains separate from the production appliance, using reviewed releases and restricted access to production secrets.

Acceptance tests for intake and scope

These are required future implementation evidence, not tests executed while writing this specification. Use sanitized fixtures, fake clocks and injected adapter failures; assert durable state and external invocation counts.

Test and trace	Required assertion
AT-01 | FR-001, DATA-001	Valid Pub/Sub delivery is durably stored before ACK; duplicate delivery creates one AUTO command. Database failure prevents ACK.
AT-02 | FR-001, FR-002	Direct BFMR mail works in both modes; approved forwards only in testing. Reject forged/other senders. Both subject layouts work without a retailer channel. Repeated same-deal links collapse; multi-deal and additional-slot mail skip.
AT-03 | FR-003, FR-005	Non-Amazon deals and bundles qualify. Submit quantity 1 for one offer; never one per component/color. No financial, payout, retailer-stock or retailer-permission gate is invoked.
AT-04 | FR-004, FR-011	Any reserved row for the same deal blocks all colors, including manual reservations. After manual release a different email may proceed; duplicate delivery of the completed email does not.
AT-05 | FR-006	Sort discovered work by original receipt time/ID. Complete failed status reads before advancing unrelated queue work. Browser operations never overlap.
AT-06 | FR-007	Persist cursor, stop process, receive new email, restart and catch up without waiting for another notification. Admit downtime email despite old receipt timestamp. Keep admitted backlog; dedupe previously completed work.
AT-07 | FR-007, CON-001	Reconnect continues the saved cursor once. No periodic Gmail polling or Inbox sweep. Expired cursor raises persistent gap and is not silently replaced, even after restart.
AT-10 | FR-010, FR-011	Review resolution submits nothing. Explicit replay is idempotent and current-admission-run only; reject unresolved review/active work. Eligible replay has fresh two-attempt budget.
AT-13 | FR-001, OPS-002	Mode switch rejects testing-forward backlog in production; direct-mail backlog remains eligible. Both configured modes create real reservations when using real adapters.
AT-22 | INT-002, SEC-002	Navigate a trusted BFMR email link without Active Deals lookup; establish stable deal identity. Reject malicious host/lookalike URLs, unapproved redirects and ambiguous forms. Unknown mapping cannot submit.

Acceptance tests for side effects

Test and trace	Required assertion
AT-08 | FR-008	Submit timeout triggers bounded reconciliation with zero automatic resubmits while UNKNOWN. Unresolved result creates review/alert; unrelated work continues after classification.
AT-09 | FR-009	Proved non-creation allows one retry only. Second failure exhausts/alerts. Closed/no-slots skip without retry/alert. No ordinal 3.
AT-11 | OPS-003	Notification failure halts new work but preserves intake. getMe/reachability do not reopen the gate. Required accepted sends do; no startup test or success alert.
AT-12 | CON-001, SEC-001	No direct reservation POST/fetch fallback. No cancellation path. Secrets are redacted. A second instance cannot own the browser.
AT-14 | FR-008, DATA-002	Crash after preparation before click, after click, after response before commit and during reconciliation. Recover using evidence; never blind-resubmit.
AT-15 | FR-009, OPS-001	Restart in RETRY_PENDING preserves ordinal 2 only. Terminal EXHAUSTED is not re-enqueued. New-run qualifying email obeys recovered evidence and live checks.
AT-16 | FR-008, FR-010	Prior uncertainty recreates current-run review. Acknowledgement alone cannot erase uncertainty. Confirmed disposition records evidence without enqueue.
AT-17 | FR-005, FR-008	Wrong quantity, wrong offer/color, possible simultaneous manual action and ambiguous bundle quantity produce CONFLICT/REVIEW. No automatic cancellation or correction.
AT-18 | OPS-003	Restart preserves Telegram block and outbox identity. Lost acceptance response can duplicate an alert but never creates another BFMR submission.
AT-19 | DATA-001, SEC-001	Database/lock loss stops new actions. Supervisor prevents old/new browsers overlapping. Stale backup restore requires reconciliation.
AT-20 | FR-004, INT-001	Verify tracker auth, account alignment, date range/buffer, all pages, IDs/status/quantity/offer mapping and immediate visibility. Incomplete reads never prove absence.
AT-21 | OPS-001, OPS-002	Ubuntu boot starts pinned headed Chromium with secured visible session. Login/MFA and restart recovery work without a physical monitor.
AT-23 | FR-012	Successful reservation remains untouched by application timers/restarts. No purchase or cancellation request. Matt releases manually after designated live tests.

Implementation plan and evidence gates

Build sequence

1. Audit actual code and repository revision; do not infer implementation status from contradictory handoff sections. Identify reuse, remove incompatible reservation request paths and capture sanitized fixtures. Pin supported runtime/library versions.

2. Implement PostgreSQL schemas, singleton ownership, durable cursor/queue, command state machine and incident outbox. Prove transaction and crash invariants with fake adapters.

3. Implement Pub/Sub pull and serialized Gmail history continuation. Remove per-restart baseline resets and receipt-time filters. Prove downtime catch-up, first initialization, watch renewal and persistent history-gap behavior.

4. Implement direct authenticated BFMR navigation, stable deal/offer identity and complete My Tracker read contract. Cover bundles and arbitrary retailers; remove business eligibility filters.

5. Implement quantity-one website submission, reconciliation, bounded retry, Telegram gates and CLI. No production write fallback or cancellation path. Complete offline failure tests before designated real reservations.

6. Package supervised Ubuntu service, secure setup, migrations, backup/restore, runbook and acceptance evidence. Release only after the gates below are satisfied.

Gate	Evidence required
G-001 BFMR contract	Sanitized browser/API fixtures prove account/deal identity, direct link resolution, form/offer mapping, bundle quantity, tracker date filters/paging, RID, release removal and immediate success visibility.
G-002 Email contract	Real sanitized MIME/auth evidence for direct and forwarded messages, both subject families, duplicate links, single bundle and multiple-deal/slot notices.
G-003 Ubuntu runtime	Actual repository/revision; pinned Chromium; secured desktop; boot, login/MFA and single-owner recovery evidence.
G-004 Catch-up boundary	Automated restart/reconnect/cursor tests; no receipt cutoff, no polling/sweep. Document expired-history limitation and obtain a separate decision if lossless long-outage recovery is required.
G-005 End-to-end	Designated real test reservation, duplicate/manual reservation check, manual release, Telegram failure/recovery, crashes and backup restore. Record versions, times, fixtures and outcomes.

Do not mark gates complete from narrative claims alone. Deliver repository location/revision, migrations, dependency lockfile, non-secret configuration example, sanitized fixtures, adapter results, operator runbook and acceptance report with this specification.

Deferred financial decisions and history

None of this page is executable MVP 1 scope. The user ultimately removed all eligibility evaluation from MVP 1. The earlier decisions are retained for traceability and possible later-MVP requirements work; no later milestone assignment or activation is implied.

Earlier clarification	Disposition after the final scope change
Reserve then evaluate	Initially approved: reserve 1, confirm creation, evaluate eligibility, release failed deals. Withdrawn for MVP 1 when all validation was removed.
Automatic failed-deal release	Initially approved through BFMR cancellation API. Superseded for MVP 1 by all releases manual. API cancellation may be revisited only in later scope.
Financial criteria	Four criteria discussed: exposure limit, total cost no greater than payout, minimum cost and maximum cost. All excluded from MVP 1.
Inclusive boundaries	Earlier accepted: exposure may equal its limit; unit cost may equal either minimum or maximum; total cost may equal payout.
Exposure source and formula	Earlier accepted: existing unpaid exposure from BFMR My Tracker, measured as expected payouts; add the proposed Amazon item cost, not its expected payout. The deliberate mixed basis was confirmed.
Other unpurchased reservations	Whether they count toward exposure was not answered. The user instead removed all validation. Remains unresolved if financial scope returns.
Extra account restriction	Rejected the handoff rule blocking a retailer account with any unpaid transaction. Earlier financial discussion allowed multiple unpaid transactions within exposure limit.
Extra retail-price restriction	Rejected blocking solely because Amazon price exceeds BFMR stated retail price. Earlier criterion was total cost no greater than payout, with other agreed limits.
Checkout and extra charges	Rejected entering checkout to obtain final total. Zero shipping, tax and other charges was a temporary MVP 1 assumption, explicitly expected to change later. Now inactive; do not automatically carry zero charges into a later purchasing MVP.
Amazon stock and bundles	Initially included Amazon in-stock and non-bundle criteria. Superseded: no stock check, all retailers and bundles permitted.
Eligible reservation and purchase	Earlier agreement left eligible reservations for manual purchase. Current MVP 1 simply leaves successful reservations untouched; no eligibility result exists.

Original roadmap: MVP 2 evaluation, MVP 3 purchasing/order update, MVP 4 shipment tracking, MVP 5 extra slots, MVP 6 multiple-deal mail. This review changes MVP 1 only; reconcile later milestones before implementing them. Prior Amazon gift requirements remain later purchasing context, not an MVP 1 browser action.

Source defects and evidence register

Finding	Required treatment
Handoff implementation status	September 26 update says reservation MVP exists, while older summary/status rows say no reservation side effects. Treat earlier claims as stale under its supersession note; verify actual code before claiming current status.
API response examples	Active Deals example depicts an object and string-only fields; handoff reports deals array, nullable closing_at/deal_information and flexible items. Not an MVP 1 discovery dependency. Preserve verified shapes if later used.
My Tracker schema gaps	Published sample is my_tracker: [{}]. Fields, pagination, date behavior, RID and immediate visibility are not proven by it. Validate before duplicate detection or retry relies on them.
API cancellation examples	Declared form encoding disagrees with JSON examples, whose quoting is also invalid; reservation cancellation uses an internal HTTP hostname. Outside MVP 1. Do not guess a production write contract from the sample.
Insurance city inconsistency	Field description allows city to be blank, but sample comment says omitted city can be required. Outside MVP 1; resolve against authoritative provider behavior if later used.
Tracker credential omission	V1.4 configuration omitted explicit BFMR read credentials while requiring My Tracker API. V1.5 requires key/secret references for tracker reads; no reservation-creation credential/endpoint is invented.
Historic live evidence	Reported successful redirect, tracker immediacy/purge and earlier tests are attributed observations, not newly validated facts. Reproduce target-environment contracts through G-001 to G-005.

Source precedence

S1: Final explicit user answers in this conversation, 9 October 2026. **S2:** BFMR_Solution_Design_Specification_v1.4.pdf, 9 October 2026, retained where not superseded. **S3:** BFMR_Automation_Project_Handoff_Updated_20260926.docx, historical implementation and design evidence. **S4:** BFMR_API.txt, supplied provider reference with known defects. **S5:** mvp.txt, original roadmap. **S6:** bfmr_deal_criteria.txt, deferred financial rules.

External implementation references

Gmail push and history: developers.google.com/workspace/gmail/api/guides/push and /guides/sync. Pub/Sub: docs.cloud.google.com/pubsub/docs/subscribe-best-practices. Telegram: core.telegram.org/bots/api. Playwright: playwright.dev/docs/api/class-page, /docs/browsers and /docs/ci. These are implementation references; provider behavior and versions require verification at implementation. No live account validation was performed to produce v1.5.

Traceability and supersession checklist

Requirement	Design pages	Acceptance evidence
FR-001, FR-002	6-7	AT-01, AT-02, AT-13
FR-003, FR-005	7-9	AT-03, AT-17
FR-004, INT-001	8-9	AT-04, AT-20
FR-006	9, 12-13	AT-05
FR-007, CON-001 intake	5-6, 10, 12	AT-06, AT-07
FR-008, FR-009	8-10, 13	AT-08, AT-09, AT-14 to AT-17
FR-010, FR-011	9-10, 14	AT-04, AT-10
FR-012	2-3, 9, 14	AT-12, AT-23
OPS-001, OPS-002	10, 15	AT-13 to AT-16, AT-19, AT-21
OPS-003	11, 14	AT-11, AT-18
INT-002, SEC-002	5, 7, 15	AT-02, AT-22
SEC-001, DATA-001, DATA-002	12-15	AT-01, AT-12, AT-14, AT-19
CON-001 UI submission	5, 8-9	AT-12, G-001

Remove these superseded MVP 1 behaviors

Windows/Edge deployment assumptions; Active Deals lookup as mandatory link resolution; Amazon-only filtering; retailer stock checks; bundle rejection; financial evaluation; one-open-transaction account gate; retail-price ceiling; timer-based or failed-eligibility cancellation; cancellation API writes; dry-run deployment default; required retailer subject channel; restart Gmail cursor reset and receipt-time exclusion.

Retain these controls

One unit and one worker; trusted email intake; correct account/deal identity; My Tracker duplicate and confirmation reads; durable dedupe/cursor/attempts; safe uncertain-result reconciliation; bounded retry; manual review; Telegram failure gating; secure headed Chromium; manual release; no purchasing; no multiple-deal or additional-slot processing.

Documentation completion versus system completion

V1.5 is the consolidated design baseline. It does not certify source-code behavior, working credentials, provider contracts, successful tests or production readiness. An implementation agent must satisfy the evidence gates and update any engineering decisions that change, with corresponding requirements and acceptance tests. Source documents remain unchanged historical references.

4. Source documents and implementation evidence

The complete v1.6 addendum follows. Its earlier sections preserve the reasoning/history; later dated corrections in the current handoff control where they differ. The project decision register, implementation plan and open issues are appended after it as supporting evidence.

Solution design v1.6 addendum

Source file: docs/BFMR_Solution_Design_v1.6_Addendum.md

Solution design v1.6 - interactive email processing and browser ownership

10 October 2026. Read alongside the unchanged v1.5 PDF. This addendum records the user-approved browser lifetime and automatic quantity-one behavior, the normal skip behavior required for duplicate/full deals, and the email-link and diagnostic corrections made during interactive testing. It affects INT-002, OPS-002, FR-002, FR-004 and SEC-002. Engineering observations below are distinguished from confirmed requirements and live acceptance evidence.

The application opens one headed Chromium window at startup, before waiting for email. It owns one persistent browser context and reuses the same page for serial deal processing throughout its lifetime. Completion of an email, a reservation, an API check or a CAPTCHA does not close or replace that context. Cookies and session state remain available between tests. BFMR can request further CAPTCHAs; one challenge per application run cannot be guaranteed. Challenges remain manual.

Only application termination closes the browser. Ctrl+C and SIGTERM initiate browser cleanup. Unexpected process termination cannot guarantee graceful cleanup. A manually closed or crashed browser must stop processing, not silently create a replacement session or retry a reservation.

The interactive email test becomes a session: after API-confirmed success it waits for the next qualifying email using the same Gmail history cursor and browser. Remaining messages from a fetched history batch are retained by the iterator and processed serially. Quantity-one reservations proceed automatically after the existing preflight checks; there is no per-reservation approval prompt. A pre-action access challenge pauses for manual completion (PAGE-READY), and each post-action check still requires CHECK-API so the operator can finish any CAPTCHA. A failed test holds processing with the window open for inspection until the operator exits. It never retries an uncertain action.

An existing reserved row found during the preflight My Tracker check is a normal skip, regardless of quantity. Log SKIPPED, submit nothing, and continue waiting for the next qualifying email in the same browser. An unknown tracker result or an uncertain post-submit outcome still holds processing for inspection.

An explicit visible BFMR full/closed/no-slots message is also a normal skip. Detect it before requiring a catalog title, and recheck if the page changes during preflight or before the reservation click. A CLOSED_NO_SLOTS result proved before submission returns normally to the email listener. Missing titles/controls alone do not prove that a deal is full, and a full page after a possible submission does not bypass tracker reconciliation.

An explicit visible `Deal not found` message on a trusted deal-detail URL is also a normal unavailable-deal skip. Log that exact reason, submit nothing, and continue waiting for the next qualifying email. Do not click `Go Back To Deals Page`, choose another deal or retry the missing deal. A hidden message or a missing catalog title alone does not establish this outcome. If a reservation action may already have occurred, reconcile through My Tracker instead of using this message as proof of non-creation.

A redirect from a deal link to the generic `/deals` listing (including `?tag=all`) is an unresolved navigation/deal-identity mismatch, not proof of a full or unavailable deal. Pause before submission and retain the browser for inspection. Never select a replacement deal from the listing or automatically replay the email. This supersedes the temporary listing-as-unavailable skip introduced during diagnosis. The user subsequently demonstrated that the supplied tracked AirPods link opens its individual deal page manually; the automated browser's reason for reaching the listing remains unproved. Visible challenges, password forms, untrusted URLs and other page mismatches retain failure/manual-handling behavior. No post-click navigation outcome bypasses My Tracker reconciliation.

BFMR email links can use HTTPS `email.mail.buyformeretail.com/c/<token>` for products, navigation and unsubscribe actions. The observed token is base64url-encoded zlib data containing URL-encoded fields; its `l` field holds the destination. Node intake decodes this locally without visiting the tracker. Accept exactly one destination field

and only an existing approved HTTPS BFMR deal-detail URL. Limit the encoded token to 8,192 characters and decompressed data to 16,384 bytes. Reject malformed, oversized or unsupported formats. Do not widen the browser navigation allowlist to include the tracking host. Navigation, image and unsubscribe links do not identify deals, and must not be followed during extraction.

Count distinct validated direct and decoded deal paths across the email. Repeated HTML/plain-text links and www aliases collapse; multiple distinct deal paths disqualify single-deal processing. Forwarded NEW DEAL subjects remain supported, including FULL RETAIL subjects without a retailer channel. Additional-slot and roundup exclusions remain in force. Skip logs distinguish subject/multiple-deal exclusions from missing or unsupported trusted deal links. Link decoding does not replace sender/provenance validation or prove browser/API account alignment.

Read the catalog title from the rendered individual deal page, rather than synthesizing it from an email subject or short-link token. The current reader waits up to 10 seconds for a unique visible semantic heading associated with the Payout Price / Retail Price section, excluding generic page and navigation labels. Missing or ambiguous titles still block reservation. This observed-layout implementation is not proof that every BFMR template has those labels. A listing route produces a navigation-specific diagnostic instead of being classified as a missing product title.

Navigation diagnostics record the requested BFMR origin/path and subsequent main-frame deal/listing origin/path changes. Omit query strings, tracking tokens, account text and unrelated route details. Failed title reads report heading counts/tags and price-label presence without logging heading text or raw pages. These diagnostics establish what the automated session visited; they do not by themselves explain a server/client redirect or justify a retry.

Verified example: the supplied AirPods tracking link decodes to `https://bfmr.com/deals/d-ikygz`, and a read-only HTTP request to that tracker returned 302 to the same destination. The actual AirPods email now qualifies as a single deal. The user's subsequent screenshot shows `Deal not found` at that exact deal-detail URL, explaining the absence of a product title in this case. The screenshot does not establish why BFMR considers the deal missing or prove a redirect to the listing; the earlier listing report is superseded as the explanation for this title failure. No reservation was made during these diagnostic checks.

This change does not certify the guided runner as the production durable intake service. It currently polls Gmail every 15 seconds and initializes its history cursor from the current mailbox profile before the startup reservation. Messages arriving during the startup reservation/CAPTCHA remain eligible. Emails already present before that baseline are excluded, including skipped mail from a previous test run; restarting does not replay them. This is a known test-runner limitation, not a change to the v1.5 durable catch-up requirement. Production cursor persistence/recovery, Pub/Sub intake and full account/deal/form identity validation remain separate work. Do not silently add an Inbox sweep or reset a production cursor to emulate this test behavior.

Acceptance evidence: `tests/browser_session.test.ts` covers one browser before intake, reuse across emails, failure retention and termination cleanup. `tests/bfmr_email_links.test.ts` covers tracked links, exclusions, repeated/multiple deals, malformed/oversized payloads and unsafe destinations using synthetic tokens. `tests/visible_deal_title.test.ts` covers title rendering, full messages, listing/login/challenge/unknown-page distinctions and sanitized diagnostics using synthetic intercepted Chromium pages. `tests/bfmr_url.test.ts` covers trusted detail/listing URL distinctions. Offline checks do not prove live multi-email/CAPTCHA reuse, all email/page templates, or end-to-end reservation correctness. See `IMPLEMENTATION_PLAN.md` for commands, results and outstanding live evidence.

Live-test startup reservation - 10 October 2026

Matt clarified that this startup behavior applies to live reservation test scripts, rather than a production entry point. Every invocation of `npm run test-real-email` (including alias `test-real-email-e2e`) and `npm run test-bfmr-ipad-reservation` opens/reuses its dedicated Chromium context, navigates to `https://www.bfmr.com/deals`, clicks the visible In Stock filter, and considers the resulting deal links in displayed DOM order. This is an explicitly authorized exception to email-triggered selection. It does not change email navigation mismatch handling. Offline tests, sign-in/read-only helpers and the unfinished-reservation recovery command do not create an extra startup reservation.

Inspect the first candidate and perform the complete My Tracker duplicate check. If already reserved or explicitly full/closed/not found, skip it and inspect the next candidate. Submit quantity one through the existing website form for the first eligible candidate. Stop candidate selection after My Tracker confirms quantity one. An empty/all-skipped list logs that no eligible startup candidate was found and proceeds to the original test. Missing titles, unknown API reads and unexplained navigation mismatches still stop processing. The iPad test may therefore create a startup reservation plus its original target reservation; its original duplicate check remains active. All releases are manual.

Startup requires no terminal CHECK-API prompt. Check My Tracker automatically immediately after submission, then reread after 10 seconds and a further 20 seconds when confirmation is absent. Before delayed reads, detected access challenges wait up to five minutes for manual completion in Chromium. This is not a CAPTCHA bypass or a guarantee that BFMR will challenge at startup. A matching API-confirmed reservation completes startup even if a page challenge remains. Normal email processing retains its existing post-action CHECK-API interaction.

A missing reservation alone never authorizes another click. Only a definitive non-creation failure plus a complete negative tracker result permits submit ordinal two, after fresh page/duplicate checks; two failed ordinals exhaust that command. The current live adapter does not prove HTTP 500 non-creation, so unproved failures remain uncertain and require review. Unknown results, quantity conflicts and unresolved submissions stop the test with its browser retained.

Persist a SUBMITTING record before any possible startup click. The local live-test snapshot defaults to `~/.local/state/bfmr/startup-reservation.json`, with transition evidence in the adjacent `.events.jsonl` file; `BFMR_STARTUP_STATE_PATH` overrides the snapshot path. Files are created with owner-only permissions and synchronized to disk. An unresolved record is reconciled through My Tracker before any new startup selection; absence alone cannot clear it. A persisted proved-failure RETRY_PENDING record resumes at ordinal two. Keep the same state path across restarts and do not delete recovery evidence to force a retry. This local journal assumes the single browser/account owner; it is not the production PostgreSQL attempt store.

The email runner captures its initial Gmail history baseline before this flow, so emails arriving while the operator completes startup remain available to the listener. Pre-baseline messages are still excluded by the test-runner limitation above.

Offline evidence: `tests/startup_reservation.test.ts` covers ordering, duplicates/unavailability, API/manual-challenge confirmation, write-before-submit, uncertain outcomes, bounded retries, persisted recovery and browser lifecycle. Synthetic Chromium fixtures in `tests/visible_deal_title.test.ts` cover clicking In Stock and preserving the resulting link order. The actual BFMR filter markup, CAPTCHA lifecycle and live startup reservation have not yet been validated. Passing offline checks does not establish those provider contracts or production readiness.

Startup filter locator correction: the user supplied a visible In Stock filter alongside a timeout from the literal single-space text matcher. The screenshot does not establish its DOM text. The locator now accepts the exact two-word label with optional whitespace, a zero-width separator or a hyphen, accommodating adjacent styled spans that expose `InStock` while displaying a visual gap. It still requires a unique visible match and uses a normal Playwright click. Synthetic fixtures exercise ordinary text, adjacent spans, nonbreaking spaces and a hyphen, with a hidden duplicate ignored. Actual BFMR DOM confirmation remains pending the next live run.

Superseding startup selection change - 10 October 2026

Matt removed the In Stock requirement after the locator still failed live. Startup now opens `/deals` and considers the visible deal links in their existing displayed order without finding or clicking any filter. Try the first deal, then advance past duplicates, explicitly unavailable deals and definitively failed reservations until a quantity-one reservation is confirmed through My Tracker or the displayed list is exhausted. Preserve the two-submit limit per deal for proved non-creation; after exhausting those ordinals, record EXHAUSTED and move to the next deal. An uncertain submission, unknown tracker read, authentication failure or quantity conflict still requires reconciliation/manual attention before another submission. This supersedes the In Stock selection and terminal exhaustion behavior described above. Startup remains manual-CAPTCHA/automatic-API verification with the same recovery journal and email baseline behavior.

Catalog title scoping correction: the startup iPhone page exposed three candidate headings (`h2`, `h5`) with both price labels. The supplied screenshot shows a catalog title, a subtitle and Additional Details. The previous reader let every heading inherit pricing context from a broad ancestor such as main/body. It now keeps candidates belonging to the tightest ancestor section containing both price labels, requires a single such section, and chooses its principal semantic heading level. Equal-level distinct titles or separate pricing sections remain ambiguous and block submission. No title is synthesized from the URL/email, and no first-heading fallback is used. Synthetic fixtures cover the pictured hierarchy; the screenshot does not establish exact live DOM nesting, so the live rerun remains necessary.

Automatic confirmation and Telegram failures - 10 October 2026

Matt instructed that CAPTCHA completion should be assumed necessary only at startup, removed CHECK-API, and requested Telegram notification when reservations fail. This supersedes every retained post-action CHECK-API requirement above. The email and iPad test flows now proceed directly from the form action to read-only My Tracker confirmation, with an immediate read and up to twelve further reads at five-second intervals while the result is NOT_VISIBLE_YET. UNKNOWN or quantity conflicts end polling without another submission. The separate recovery helper also no longer requires CHECK-API; its explicit pre-action identity-review PAGE-READY step remains.

Startup still waits for manual CAPTCHA completion in Chromium and uses its existing automatic confirmation timings. No CAPTCHA bypass is added. After startup the email/iPad preflight treats a detected access challenge as an unexpected failure and alerts, without another terminal readiness prompt. Post-action confirmation remains automatic even if a challenge appears; API confirmation is authoritative. An unconfirmed possible submission remains uncertain and blocks additional work. BFMR may still issue challenges despite the startup-only operating assumption.

Live test entrypoints require TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID before opening the reservation flow. The server-side sender uses the official Telegram Bot API sendMessage method with a bounded request timeout and validates the HTTP response, ok flag and message identifier. It does not log token-bearing URLs, response bodies or underlying transport exceptions. Alerts contain the runner/deal context and direct the operator to My Tracker/the terminal; raw emails/account pages are not sent. Email/startup processing failures and iPad/recovery failures send an alert before the existing stop/inspection path. Proven startup submission failures also alert before retrying/advancing. Normal duplicate/unavailable skips do not generate email/startup failure alerts.

Alert-delivery failure is logged explicitly and does not authorize further reservation processing. These test-runner alerts are immediate sends, not the production durable PostgreSQL notification outbox; failed deliveries require operator intervention and are not automatically replayed across restarts. The existing production outbox requirement remains outstanding. No test alert or live Telegram message was sent during implementation. Synthetic injected-transport tests verify request construction, required configuration, delivery validation, redacted failures and preserving the original reservation failure; fake confirmation tests verify polling without a prompt or resubmission.

Provider reference: Telegram sendMessage.

Decision register

Source file: docs/DECISIONS.md

Decision register

Final user decisions from 9 October 2026. This is a summary of v1.5, not a substitute for it.

Topic	Final decision
Starting point	Fresh implementation in this repository; no reuse of the earlier codebase
Platform	Ubuntu with Playwright bundled Chromium in headed mode
Trigger	A single new-deal email; no required retailer channel in subject
Retailers	All retailers; Amazon-only restriction withdrawn
Bundles	Included; quantity 1 refers to the BFMR offer
Additional slots / multiple deals	Excluded from MVP 1
Business eligibility	No financial, stock, profitability or retailer-eligibility evaluation
Duplicate protection	Check My Tracker before reserving; any active reservation for the same deal blocks creation
Navigation	Open trusted email deal link directly in dedicated authenticated browser; no mandatory Active Deals API resolution
Reservation creation	Playwright website form only; no reservation-creation API
Confirmation	My Tracker API
Testing	Real reservations in testing and production modes
Release	Entirely manual; no one-minute or failed-eligibility release
Purchasing	Outside MVP 1
Downtime	Catch up and process qualifying missed emails using durable cursor and dedupe

Superseded intermediate choices

The conversation temporarily expanded MVP 1 to reserve, evaluate financial/stock/bundle criteria and cancel failed deals through the cancellation API. The user later removed all eligibility evaluation and restored manual release. Do not implement those intermediate choices.

The final removal of the Amazon-only restriction also removes the need for an API retailer eligibility check. It does not remove the My Tracker duplicate or confirmation checks, or operational safety checks.

Deferred financial discussion

Page 19 of v1.5 records earlier clarifications: inclusive limits, the mixed exposure calculation, rejected retailer-account and retail-price restrictions, and the temporary zero-extra-charges assumption. None is active MVP 1 scope, and none automatically becomes an approved later-MVP requirement. The exposure treatment of unpurchased reservations was not answered.

Retained design versus user decisions

The database schema, retry timings, initial Gmail baseline, local CLI and other engineering details are specified in v1.5. Preserve their distinction from confirmed user requirements. In particular, the expired-history recovery limitation remains explicit in `OPEN_ISSUES.md`.

10 October 2026 - browser lifetime

User decision: open Chromium when the application starts; reuse that window for all processing and tests; close it only when the application terminates. See v1.6 addendum. CAPTCHA completion is manual, and reuse does not guarantee BFMR will never challenge again.

10 October 2026 - automatic quantity-one authorization

User authorized quantity-one reservations without per-reservation approval. Remove RESERVE-1 from the email runner and iPad live test. Retain preflight/duplicate checks and manual access-challenge completion; CHECK-API remains a readiness prompt after the action.

10 October 2026 - BFMR tracked email links

Observed integration format: the forwarded AirPods new-deal email wraps product and footer links in HTTPS `email.mail.buyformeretail.com/c/<token>`. The token is base64url-encoded zlib data containing URL-encoded fields; `l` contains the destination. Node intake decodes this locally with bounded input/output sizes and accepts exactly one `l` field only when its value passes the existing trusted BFMR deal URL validator. No tracker, footer or unsubscribe URL is visited. The browser destination allowlist is unchanged. Unsupported/malformed tokens do not identify a deal. Multiple distinct direct/decoded deal paths prevent single-deal qualification; repeated links collapse.

10 October 2026 - live-test startup reservation

Matt corrected “production” to “every test script”: the live reservation test entrypoints (`test-real-email`, its E2E alias and `test-bfmr-ipad-reservation`) perform startup on every invocation. Open `/deals`, click In Stock, reserve quantity one of the first unreserved available deal in displayed order; duplicate/unavailable candidates advance to the next deal. Matt completes CAPTCHA manually in Chromium. Automatically verify startup using My Tracker without CHECK-API and preserve existing conservative retry limits. This startup exception does not add real reservations to offline tests, read-only/sign-in helpers or unfinished-reservation recovery. See the v1.6 addendum for recovery evidence and current live-validation gaps. Normal email post-action CHECK-API remains unchanged.

10 October 2026 - remove startup In Stock filtering

Matt explicitly removed In Stock selection. Try the first displayed deal on `/deals` and advance to later displayed deals after duplicates, unavailability or proved failure until My Tracker confirms success. Retain the existing two-submit limit per deal, advance after proved exhaustion, and reconcile uncertain actions before any additional submission. This supersedes the filter selection in the earlier startup decision.

10 October 2026 - no CHECK-API; Telegram failures

Matt instructed the app to assume CAPTCHA is completed only at startup, remove CHECK-API and notify him through Telegram if a reservation fails. All live reservation/recovery post-action API checks are now automatic. Startup remains manual CAPTCHA in Chromium; unexpected later challenges stop/alert instead of requiring PAGE-READY in the email/iPad flow. Unknown/unconfirmed outcomes still cannot authorize resubmission. Configured Telegram failure alerts are required for live entrypoints; immediate test-runner delivery is distinguished from the production durable outbox in v1.6.

MVP 1 implementation plan

Baseline: Solution Design Specification v1.5, especially pages 16-18 and 21. M0 project/bootstrap scaffolding is complete. M1 orchestration scaffolding exists, but production-grade durable persistence and the remaining production milestones are unfinished. The “initial handoff” refers to the documentation-only starting repository, not the current working tree.

M0 Establish the fresh project [COMPLETED]

- Read the baseline, decisions and open issues. Fresh implementation without legacy code.
- Pinned supported Node.js 22, TypeScript 5.7, Vite 6.2, Playwright and PostgreSQL 16 schema.
- Established project structure (`src/db`, `src/domain`, `src/adapters`, `src/orchestrator`, `tests`), reproducible scripts (`npm test`, `npm run typecheck`, `npm run build`, `npm run dev`), package-lock, and non-secret `.env.example`.
- Documented adapter interfaces (`IGmailAdapter`, `IMyTrackerAdapter`, `IBrowserAdapter`, `ITelegramAdapter`) and safe local configuration.
- Captured implementation evidence index keyed by AT/G IDs.

Completion evidence:

- Setup verified on clean checkout: `npm test`, `npm run typecheck`, and `npm run build` pass cleanly.
- Zero secrets tracked in Git.

M1 Orchestration scaffolding [PARTIAL]

- Added PostgreSQL schema/migration and TypeScript interfaces as scaffolding. The current `src/db/database.ts` implementation stores state in process-local Maps and uses an in-process lock; it is not a PostgreSQL-backed database and does not provide ACID transactions, cross-process ownership, or restart durability.
- Added an in-memory orchestration model for the T1–T6 transaction boundaries, retry/review states, duplicate checks, and notification gating. These are simulated state transitions, not durable commits around real provider side effects.
- Added fake Gmail, My Tracker, browser, and Telegram adapters for offline testing. Fake-adapter coverage verifies modeled invariants only; it does not prove provider behavior, persistence, or crash recovery.

Scaffolding evidence:

- `tests/m1_orchestration.test.ts` exercises the in-memory model and its failure cases.
- The earlier 11-test result is historical scaffolding evidence, not acceptance of production M1. Current required M1 work remains: implement and migrate the PostgreSQL store; make T1–T6 atomic and durable; implement cross-process singleton ownership; prove restart recovery, cursor preservation, and Telegram outbox behavior against PostgreSQL; then record the resulting test and migration evidence.
- No production PostgreSQL persistence or live BFMR operation is established by this scaffolding.

M2 Gmail and Pub/Sub intake

- Implement watch renewal, StreamingPull, persistence before ACK, serialized history reads and email classification.
- Preserve cursor across restart; perform startup/reconnect continuation. Keep watch baseline separate from consumption cursor.
- Support both subject families, direct/testing-forward sender policy and single-deal classification.
- Treat expired/missing established cursor as an explicit fault; no silent reset or Inbox sweep.

Exit criteria: AT-01/02, AT-06/07 and intake portions of AT-13 pass. G-002 fixtures and G-004 recovery evidence are recorded.

M3 Validate BFMR read contracts

- Implement dedicated authenticated browser navigation and stable deal/offer identity extraction.
- Implement My Tracker read adapter with secure credentials, verified account alignment and complete date/pagination handling.
- Validate bundles, color variants, duplicate detection and reported reservation visibility/release behavior using sanitized evidence.
- Resolve documented contract gaps from actual observations; do not guess field names or guarantees.

Exit criteria: AT-04/20/22 and read portions of G-001 pass. Record which live claims remain unproven and prevent unsafe submission/retry paths from using them.

M4 Reservation and operational controls

- Implement one-unit website-form submission, reconciliation, one proved-failure retry and review/exhaustion blocks.
- Implement Telegram acceptance/recovery gates and local status/review/reprocessing CLI.
- Include arbitrary retailers and bundles without business eligibility evaluation. Add no cancellation or purchasing path.
- Complete offline side-effect failure tests, then designated real reservation tests under the documented gates.

Exit criteria: AT-03, AT-08 to AT-18, AT-20, AT-22/23 pass with appropriate offline/live evidence; G-001 and relevant G-005 records are complete.

M5 Ubuntu deployment and acceptance

- Package service startup, exclusive process-group shutdown, headed Chromium desktop access, secure configuration and logs.
- Implement off-host encrypted backups, restore procedure, disk/watch monitoring and operator runbook.
- Demonstrate reboot, login/MFA recovery, Telegram failure and stale-backup reconciliation.
- Assemble the complete acceptance report and identify any unresolved limitations before enabling unattended operation.

Exit criteria: AT-19/21, G-003 and G-005 pass; all applicable acceptance IDs are accounted for. No acceptance item is marked complete solely from a narrative claim.

Progress updates

For each milestone record status, commit, commands run, test outcomes, sanitized fixture references, remaining gaps and any changed design decision. Keep this file current as implementation proceeds. Consult v1.5 for detailed test assertions; the milestone grouping does not replace them.

10 October 2026 — reusable interactive test session

The email E2E runner now owns Chromium for its entire session, processes successive emails serially with one history iterator, and holds failed tests open for inspection until exit. Ctrl+C/SIGTERM clean up the browser. See the v1.6 design addendum. Lifecycle regression tests use fakes; live multi-email/CAPTCHA reuse is not yet verified. Existing production intake and visible-title extraction gaps remain.

10 October 2026 — visible catalog title preflight correction (M3 / EV-001 / G-001)

The paused email test used an immediate `h1:visible` lookup although the supplied screenshot showed a catalog title. The shared browser adapter now waits up to 10 seconds for a unique visible semantic heading (`h1`–`h6` or `role=heading`) associated with the Payout Price / Retail Price section. Navigation and generic page labels are excluded; missing or ambiguous titles still stop preflight. This fixes both live runners that call `getVisibleDealTitle`; no email-subject or URL-title fallback was added.

Evidence: `tests/visible_deal_title.test.ts` uses synthetic local HTML in a fresh headless Chromium instance to cover alternate heading levels, whitespace normalization, delayed rendering, hidden/missing headings and ambiguity. `node --import tsx --test tests/**/*.test.ts`: 30/30 pass. `npm run typecheck` and `git diff --check` pass. The direct Node test entry point avoids the sandbox restriction on the tsx CLI's IPC socket; fixture Chromium tests ran outside the sandbox. No BFMR requests or reservations were made. The screenshot does not establish the actual DOM tag; live markup validation and the remaining EV-001 identity/form evidence are still outstanding.

10 October 2026 — duplicate preflight continues intake (FR-004 / AT-04)

User requested that an already-reserved deal should keep the app waiting for additional deals. The interactive email runner now logs SKIPPED and returns normally for both CONFIRMED and EXISTING preflight results, so the session advances to its next email without a form submission. UNKNOWN reads and unresolved submission outcomes retain the existing pause behavior. This matches the baseline normal duplicate skip; it adds no replay or retry. Validation: TypeScript check (including the runner), offline session/tracker tests, and diff whitespace check; no live reservation attempted.

10 October 2026 — full deals continue intake (FR-003 / AT-03)

The email runner now checks for the screenshot's visible `Deal Full - Looks like you missed this one` message before title extraction and again after the title wait. Full/closed/no-slots results log SKIPPED and return to the same email iterator and browser. The browser adapter shares the explicit visible-message detector, checks again after waiting for quantity controls and before clicking, and only returns CLOSED_NO_SLOTS before a submission. Missing controls and uncertain outcomes remain failures; no post-click shortcut or retry was added.

Evidence: synthetic Chromium fixtures recognize the full page without title/prices, recognize closed/no-slot messages, and reject hidden messages, loading pages and retailer stock wording. `node --import tsx --test tests/**/*.test.ts`: 31/31 pass. `npm run typecheck`, explicit TypeScript checking of `scripts/test_real_email_e2e.ts`, and `git diff --check` pass. No live BFMR requests or reservations were made.

10 October 2026 — tracked email destination extraction (FR-002 / SEC-002 / G-002)

Added the Node-only `bfmrEmailLinks` intake parser for the E2E runner, other email scripts, and orchestration engine. It decodes the observed BFMR tracking format locally and passes only validated direct deal destinations to the shared parser. The shared parser now counts distinct deal paths, rejects multi-deal bodies, and collapses repeated direct/decoded links. Skip logs distinguish unsupported/missing trusted links from nonqualifying subjects. The frontend parser remains free of Node compression dependencies.

Read-only evidence: the actual forwarded AirPods email now returns `isSingleDeal=true`, `isMultiDeal=false` and the direct BFMR deal URL. Only Gmail reads were made; no tracking URL, browser form or reservation endpoint was called. No raw email, recipient metadata or tracking token was saved to the repository or diagnostic output. Synthetic fixtures cover product plus navigation/unsubscribe links, forwarded subject, quoted-printable soft wrapping, repeated links, multiple distinct deals, exclusions, malformed/oversized tokens and unsafe/lookalike destinations. `node --import tsx --test tests/**/*.test.ts`: 36/36 pass. `npm run build`, explicit runner TypeScript checking, and `git diff --check` pass. Other tracking encodings remain unsupported; this is not proof of all email templates or production intake readiness.

10 October 2026 — listing redirect during deal preflight

User reported that the paused Amazon Direct email had reached `https://www.bfmr.com/deals?tag=all`. This explains the absent product title; it does not establish that BFMR marked the deal full. The interactive runner now skips a trusted exact deals-listing redirect before submission, including redirects during title rendering or page confirmation. It logs the listing reason, submits nothing, never selects a replacement deal, and returns to listening. Challenges, login forms, untrusted destinations and other page mismatches still pause. Post-click reconciliation is unchanged. Added bounded heading-count/tag/label diagnostics for actual title-extraction failures without logging heading text or raw page data.

Evidence: synthetic intercepted Chromium pages exercise listing, challenge, password form, account-page mismatch, untrusted host and normal deal-title reading. `node --import tsx --test tests/**/*.test.ts`: 39/39 pass. Project and explicit runner TypeScript checks and `git diff --check` pass. No live reservation or retry was made. Public unauthenticated markup reads returned an HTTPS canonical redirect and 403; the actual account DOM was not accessible, so the listing observation comes from the user.

10 October 2026 — correction: listing redirects do not prove unavailability

Supersedes the preceding listing-skip behavior. The user supplied the exact tracked AirPods link and confirmed manual navigation reaches the individual deal. Local decoding and a read-only tracker request both confirmed destination `/deals/d-ikygz` (HTTP 302). The automated listing transition's cause remains unproved. Removed listing redirects from normal unavailable skips; they now pause as navigation/identity mismatches. Added requested-page and main-frame path diagnostics, omitting queries/account data, and a specific listing error around title extraction. Explicit full/closed messages and preflight duplicates continue normally. Updated `BFMR_Solution_Design_v1.6_Addendum.md` with the final behavior, tracked-link format/limits, intake baseline limitation and outstanding evidence. No reservation or retry was made.

Validation: `node --import tsx --test tests/visible_deal_title.test.ts tests/bfmr_url.test.ts`: 13/13 pass. Project and explicit runner TypeScript checks and `git diff --check` pass. The new diagnostic logger requires restarting the runner; the failed run's preceding navigation output is still needed to establish its actual route sequence.

10 October 2026 — confirmed Deal not found page

The user's screenshot shows the correct AirPods short-link URL with an explicit `Deal not found` message. This explains the title failure without assuming a listing redirect or tracking-decoder fault. Added that exact visible message to the shared unavailable-page detector, with its own log reason. The existing preflight and pre-click unavailable checks now skip it and continue intake; no back button, alternate deal or retry is used. Hidden messages and unexplained listing routes remain inconclusive. Updated the v1.6 addendum with the screenshot evidence and final behavior. Synthetic intercepted Chromium coverage checks the exact missing-deal layout, unchanged URL and hidden-message rejection.

Validation: `node --import tsx --test tests/**/*.test.ts`: 40/40 pass. Project and explicit runner TypeScript checks and `git diff --check` pass. No live reservation or retry was made.

10 October 2026 — live reservation test startup

Connected the shared startup flow to the real-email and iPad live reservation test entrypoints. It filters the listing through the website, skips duplicate/unavailable candidates, submits quantity one and confirms automatically through My Tracker. Startup has no CHECK-API prompt. Conservative retry/recovery rules use an owner-only fsynced local snapshot plus transition journal; this is test recovery evidence, not the production PostgreSQL store. Gmail baseline capture now precedes startup so arrivals during CAPTCHA are retained. Updated v1.6 design and decision register with the user's production-to-test clarification.

Evidence: full offline suite initially passed 52/52; subsequent targeted startup checks cover manual challenge completion and the finalized immediate/10s/30s API checks. Build, project TypeScript and explicit live-entrypoint TypeScript checks pass. Synthetic Chromium verifies In Stock selection and resulting deal order. No live reservation scripts were run for validation. Actual BFMR filter markup and manual CAPTCHA/startup confirmation remain pending live acceptance; account alignment and production durability gaps remain open.

10 October 2026 — startup In Stock label matching

The user's screenshot shows In Stock while the literal-space locator timed out. Changed the exact label matcher to tolerate absent/alternate whitespace and hyphens without matching unrelated wording. The DOM cause is inferred, not proven by the screenshot. Added synthetic Chromium cases for adjacent styled spans (`InStock` text), nonbreaking spaces, hyphens and a hidden duplicate alongside the original label/order check. TypeScript and whitespace checks pass; live BFMR rerun is still required. No live form action was performed during diagnosis.

10 October 2026 — unfiltered startup progression

Removed In Stock lookup/click entirely following Matt's live feedback. Both live test entrypoints now use `getStartupDealUrls` to read the existing visible listing order. Proved failure exhaustion advances to the next displayed candidate instead of terminating selection. Uncertain outcomes still block further submissions. Updated synthetic coverage to verify order with and without a filter, no filter click, and two proved failures followed by successful reservation of the next candidate. Project TypeScript and targeted startup checks pass; live validation remains pending. No live BFMR mutation was performed.

10 October 2026 — pricing-scoped catalog title

Fixed broad ancestor pricing association that admitted subtitles and Additional Details headings as catalog titles. Title candidates now require a single tightest pricing section and a unique principal semantic heading within it. Synthetic Chromium fixtures cover the screenshot's title/subtitle/details structure, same-level ambiguity and separate pricing sections. Project TypeScript checks pass; real page DOM validation remains pending. No live reservation attempted.

10 October 2026 — automatic API confirmation and live Telegram failure alerts

Removed CHECK-API from the email, iPad and Watch recovery scripts. Shared read-only confirmation polls delayed visibility automatically and stops on unknown/conflicting results. Unexpected post-startup preflight challenges stop/alert in the email/iPad tests. Added server-side Telegram failure reporting using existing configured environment variables, validated responses and sanitized transport exceptions. Startup proved failures notify before progressing; unresolved failures notify before existing failure handling. Notification failure stops further processing and is explicit locally. This is immediate live-test reporting; production durable outbox/replay remains separate work. No live Telegram message or BFMR reservation was sent during implementation.

Evidence: injected-transport notification tests cover missing configuration, valid delivery, rejected/malformed responses, credential redaction, original-error retention and no success alert. Fake confirmation tests cover delayed visibility, bounded polling and immediate unknown/conflict termination; startup coverage verifies notification before retry and halt on delivery failure. Project and all three explicit script TypeScript checks pass. Live BFMR/Telegram delivery acceptance remains pending.

Final validation for this slice: `node --import tsx --test tests/**/*.test.ts` passed 59/59 tests, including synthetic Chromium and injected Telegram transport. Project and explicit live-script TypeScript checks and `git diff --check` passed. No live Telegram messages or BFMR mutations were made.

Open issues and setup inputs

Source file: docs/OPEN_ISSUES.md

Open issues and implementation inputs

These items do not reopen settled MVP 1 scope. Separate a product decision from an external fact that must be verified and from a setup input.

OQ-001 Recovery after Gmail history expires

Type: unresolved requirement boundary.

The user requires catch-up after downtime. V1.5 implements this using the saved Gmail history cursor while retaining the earlier prohibition on Inbox sweeps. If Gmail rejects an expired cursor, the specified behavior is to preserve the gap, alert and stop new intake. Restart cannot silently reset it.

This does not guarantee catch-up after arbitrarily long downtime. If that guarantee is required, Matt must decide whether a bounded recovery Inbox scan is permitted. Implement the documented stop-and-alert behavior in the meantime; do not silently broaden access or claim lossless recovery.

EV-001 BFMR browser identity and quantity

Type: integration evidence, G-001.

Prove direct email-link navigation, expected account identity, canonical deal_id extraction, correct form/offer association, bundle quantity-one meaning and variant selection. Unknown identity cannot submit. Historical observations are not target-environment test results.

EV-002 My Tracker contract

Type: integration evidence, G-001.

Verify required fields, pagination completion, date-filter semantics/timezone, RID, deal/status/quantity/offer mapping, account alignment, active-reservation coverage, immediate visibility and manual-release removal. The supplied reference's empty-object response sample is insufficient.

If immediate visibility cannot be proved, an empty result after an uncertain click cannot authorize automatic resubmission. Do not rely on a 24-hour date window if evidence shows active reservations can be older.

EV-003 Email authentication and templates

Type: integration evidence, G-002.

Obtain sanitized direct and forwarded MIME/authentication examples. Verify trusted Gmail evidence, link preservation, both subject formats, repeated links, bundles versus multiple distinct deals, and extra-slot notices.

SETUP-001 Secure service configuration

Type: environment input.

Required: Gmail account/OAuth setup, Google project/topic/pull subscription, BFMR expected account and API credentials, dedicated browser login profile, Telegram bot/private chat, Ubuntu desktop/service access, PostgreSQL and backup destination. Supply secrets through secure local configuration, never Git or chat.

SETUP-002 Versions and operational evidence

Type: engineering setup and validation.

Pin compatible runtime/dependency versions during implementation. Validate headed browser boot/login, singleton ownership, encrypted backups and restore. Repository creation and the PDF do not establish those capabilities.

Deferred items

Financial evaluation, purchasing, cancellation API contracts, shipment/payout handling, multi-deal mail and additional-slot notifications are outside MVP 1. V1.5 page 19 preserves earlier discussion without activating it.

Current amendments to v1.6

Controlling update for implementation continuation | 10 October 2026

How to read this update. This section supplements the consolidated v1.6 handoff. It records later user decisions, current implementation behavior, and evidence. Where this section conflicts with earlier baseline or historical notes in the preserved pages, this section controls. The Markdown addendum and implementation plan remain the detailed source records.

1. Project state and current scope

M0 is complete as project/bootstrap scaffolding. M1 is scaffolding only; production persistence remains unfinished. The local journals and live test flows described here are recovery scaffolding, not the PostgreSQL ACID orchestration store, durable production Gmail intake, transactional outbox, or multi-process account lock. M2–M5 remain incomplete; use “Exit criteria” in the implementation plan as criteria, not completion claims.

- MVP 1 email flow: validate one qualifying BFMR deal email, navigate the trusted deal URL in the expected BFMR account, check My Tracker, reserve quantity one through the visible BFMR form, and confirm through complete My Tracker reads. Then check the exact new reservation for an Amazon supplier. Keep it if Amazon is listed; otherwise cancel that exact reservation and verify release. Unknown evidence stops and alerts.
- After duplicate, full/closed/no-slots, explicit Deal not found, or a proved failed candidate, resume listening for the next qualifying email. A deals listing redirect, missing/ambiguous title, unknown API response, or uncertain submission is not proof of unavailability and must stop for review/reconciliation.
- Accept all retailers and bundles in incoming mail. Skip multiple distinct deals and additional-slot/roundup notices. Decode supported BFMR email tracking links locally and permit navigation only to a validated HTTPS BFMR deal-detail URL.
- All live test runners that invoke startup perform the authorized startup reservation except the Watch recovery helper. Startup walks displayed deal links in order without the In Stock filter. Once My Tracker confirms the quantity-one startup reservation, cancel that exact reservation and verify release before continuing. Startup reservation is only for CAPTCHA; do not run Amazon supplier evaluation on it.
- Startup retries/advances only on proved non-creation plus complete negative tracker evidence, with fresh page and duplicate checks. At most two submit ordinals per candidate. Unknown results and uncertain clicks require reconciliation; never blindly retry.
- CAPTCHA is completed manually in Chromium. API confirmation is automatic; CHECK-API prompts were removed. Telegram alerts are required for reservation/test failures; delivery failure stops further processing. Normal duplicate/unavailable skips are not alerts.

2. BFMR evaluation and cancellation

Incoming email reservations

After confirming a newly created quantity-one reservation, join the exact My Tracker reserve_id and deal_id to the complete active-reservation API data. Inspect supplier entries for Amazon. A positive Amazon supplier label retains the reservation for the user to manage. If no Amazon supplier is present in complete, validated evidence, cancel only that exact new reservation and verify its release. Do not infer Amazon presence from the email, product title, or retailer URLs. Malformed, incomplete, conflicting, or unreadable supplier data is UNKNOWN and must stop and alert; it is never treated as Amazon absence.

Startup-only cleanup and cancellation contract

The user approved testing form-encoded cancellation. Send one HTTPS POST to `https://api.bfmr.com/api/v2/my-tracker/reservation/cancel`, with `Content-Type: application/x-www-form-urlencoded` and one `reserve_id[]` value. Before the write, require the exact confirmed reservation/deal IDs, Reserved status, and quantity one in a complete tracker read. Reject redirects. Verify release from complete tracker data or explicit canceled status; HTTP success alone is not confirmation. An uncertain POST is reconciled read-only, never automatically repeated.

Persist evaluation/cancellation intent before the write. The owner-only test journal can recover pending cleanup after restart; it is not the unfinished production database. The sole designated live form-encoding test reservation (Google Fitbit Air Obsidian) was canceled and a subsequent complete tracker read confirmed release. No other reservation was canceled in that test.

3. Amazon checkout rehearsal

The standalone rehearsal is separate from BFMR email intake. The user selected `https://www.amazon.com/dp/B09JQMJHXY`, quantity one, Amazon's saved default delivery address, and a stop at the final order review. No order submission is authorized.

- Run `npm run test-amazon-checkout`. It first completes the BFMR startup reservation, My Tracker confirmation, exact cancellation and release verification, then opens a separate Amazon tab in the same persistent Chromium context/profile. One runner owns that profile.
- Open and validate the requested product page before cart preflight. Amazon login, MFA and challenges are completed manually in Chromium. The runner polls for completion (up to ten minutes), then automatically restores the requested product/cart route, including after Amazon redirects to its home page. It no longer waits for terminal Enter after authentication.
- Before Add to cart, inspect the active cart. Add exactly one target only if the cart is confirmed empty. Reuse a cart that contains exactly the target at quantity one. Unknown markup, other items, multiple lines, wrong quantity, or identity mismatch stop without cart mutation. Never repeat an uncertain Add to cart action.
- A verified target-only cart can proceed through checkout. Address/payment choices remain manual; use the saved default address and an existing payment method. Final review requires the target ASIN, quantity one, order total, and visible Place your order control. Observe this screen and stop. Never click Place your order, Buy Now, or a trial/subscription control.
- The cart preflight previously stopped before product navigation when Amazon's cart markup did not match the fixture. The updated runner opens the product first; if cart state is still unknown, it returns to the product, logs sanitized structural counts, and stops before adding. This keeps the product available but does not resolve the live cart markup.

Current live limitation: The user reported "Amazon cart contents could not be verified; no item was added." Offline fixtures recognize several empty-cart messages and exercise unknown-layout handling, but the real authenticated cart DOM has not been inspected. Therefore the current version is expected to reach and leave the requested product page for inspection, but may still stop before adding anything until its cart layout is identified and supported.

4. Browser and email continuity

- The BFMR email runner keeps one headed Chromium context open and listens serially for further qualifying messages. A handled duplicate/full/deal-not-found skip returns to the listener; no automatic replay of an uncertain reservation occurs.
- Live test startup captures its Gmail history baseline before startup reservation, so mail arriving during CAPTCHA/startup remains eligible. This runner initializes from the current mailbox profile; pre-baseline messages, including skipped mail from a previous run, are excluded. This is a known test-runner limitation; production durable cursor/catch-up remains unfinished.
- The user confirmed CAPTCHA is expected only at startup. Unexpected later access challenges are failure/alert conditions. Do not bypass authentication challenges or choose a replacement deal from a listing page.
- Use the same configured Chromium profile across BFMR and Amazon. Browser/profile ownership is single-runner. Failures retain the browser for inspection; normal termination closes it.

5. Validation evidence and limits

Recent automated evidence

- The latest Amazon checkout regression suite passed 12/12 synthetic intercepted Chromium tests, covering product-first navigation, unknown-cart recovery, empty-cart heading variants, login/MFA detours through Amazon home, authentication timeout, cart isolation, quantity/product verification, manual checkout and zero order submissions.
- The project TypeScript check and strict targeted TypeScript check passed after the Amazon product-first/cart-preflight update; git diff --check passed.
- Prior implementation-plan entries record synthetic coverage for tracked BFMR links, deal title scoping, full/not-found distinctions, startup candidate progression and journals, My Tracker polling, Telegram transport behavior, exact supplier evaluation and cancellation recovery. Consult the current implementation plan for the recorded command/result of each slice.

What these tests do not prove

Synthetic pages do not establish the current Amazon authenticated cart markup, Amazon checkout/provider behavior, live MFA completion, real delivery/payment selection, or a successful final review. They do not prove BFMR API contracts beyond the explicitly recorded single cancellation test, all message/page templates, production durability, live Telegram delivery, or production readiness. Do not claim an Amazon order was placed; none was submitted.

6. Continuation checklist for the next coding agent

- First resolve the live Amazon cart-layout mismatch from a sanitized DOM snapshot or visible markup. Add a narrow, evidence-backed cart parser and synthetic fixture. Preserve product-first navigation and stop safely when cart contents remain unknown.
- Continue M1 production durability: PostgreSQL schema/transactions, idempotency, inbox cursor and queue atomicity, durable attempt journal, account lock, cancellation/evaluation recovery, transactional notification outbox, backup/restore and operational runbooks.
- Complete external contract verification for Gmail, BFMR URL/form/My Tracker and supplier data, cancellation edge cases, Telegram delivery, and account identity. Clearly distinguish offline fixture evidence from live acceptance.
- Keep the v1.5 PDF baseline and historical text intact. This appended dated amendment is the latest controlling handoff. Maintain the Markdown addendum and implementation plan alongside this PDF as the editable working record.

Controlling detailed sources: docs/BFMR_Solution_Design_v1.6_Addendum.md, docs/IMPLEMENTATION_PLAN.md, docs/DECISIONS.md, docs/OPEN_ISSUES.md, README.md, AGENTS.md and the source/test tree. These are repository-local paths and should be read from the working tree.